



Contexte et problématique

- Association musicale (chorale en pupitres : soprano, alto, ténor, basse)
- Diffusion de partitions et audios pour répétitions
- Échanges réalisés par email

Difficultés

- Absence de version de référence
- Perte d'informations
- Manque de traçabilité
- Difficulté d'accès pour certains utilisateurs
- Pas de gestion différenciée des droits



Organisation du projet et méthodologie

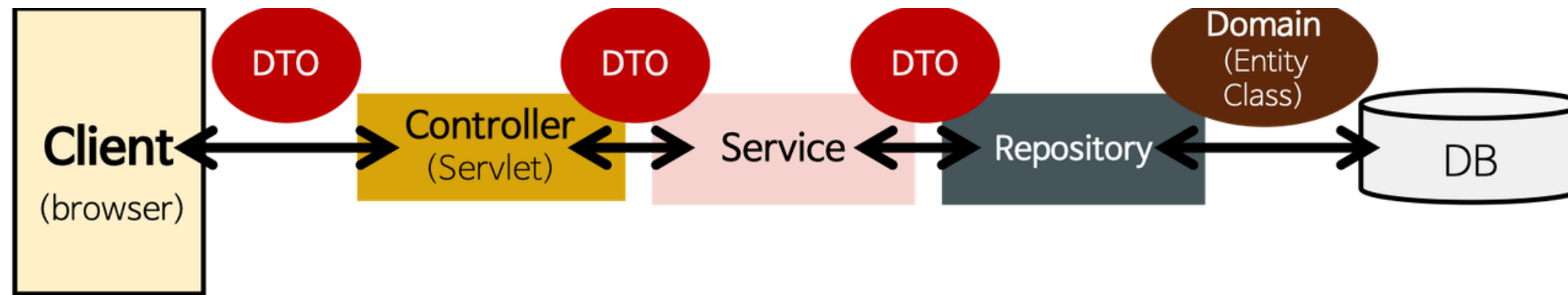
Contexte

- Besoin évolutif
- Attentes fonctionnelles progressives
- Méthodologie Agile inspirée Scrum
- User Stories
- Priorisation MoSCoW

Objectif

- Livraison priorisée des fonctionnalités essentielles

Architecture technique



Client <-dto-> controller(web) - service - repository(dao) <-domain(entity)-> DB

Backend

Développé en Java avec Spring Boot

- Architecture multicouche
- Logique métier et sécurité centralisées
- Persistance via JPA + PostgreSQL
- Authentification stateless par JWT
- Utilisation de DTO pour sécuriser les échanges
-

Frontend

Application web monopage (SPA)

- Gestion de l'affichage et des interactions
- Aucune logique métier côté client
- Séparation stricte frontend / backend



Conteneurisation, déploiement et outils

Conteneurisation

- Docker

Gestion de version

- GitHub

Documentation API

- Swagger

Outils de base de données

- DBeaver
- Insomnia



Rôle et responsabilités

Projet réalisé en binôme

- **Mon rôle : conception et développement backend**
- Analyse des besoins et modélisation des données
- Architecture applicative et règles métier
- Implémentation des API REST
- Gestion des transactions (invitations, rattachements)
- Rédaction et exécution des tests backend



Analyse des besoins

Profils utilisateurs et besoins clés

- **Cheffe de chœur**
- Crée et organise les ensembles et morceaux
- Gère les membres et leurs rôles
- Besoins : outil simple, centralisé, sans emails
- **Membre / Choriste**
- Consulte partitions et suit les morceaux
- Reçoit notifications et propose des documents
- Besoins : accès facile et sécurisé aux documents
- **Administrateur**
- Supervise la plateforme et résout les problèmes techniques
- Besoins : cohérence des données, maintenance et support



Objectifs fonctionnels & techniques

Objectifs fonctionnels

- Créer et gérer des ensembles musicaux
- Gérer l'appartenance des utilisateurs aux ensembles
- Partager les documents
- Tracer les actions clés

Objectifs techniques

- Sécuriser l'accès aux données
- Empêcher les modifications non autorisées
- Assurer l'intégrité des relations entités
- Permettre l'évolution du système



Organisation du projet

Méthode

- Approche itérative
- Découpage en fonctionnalités indépendantes
- Priorisation des éléments structurants

Suivi

- 1 fonctionnalité = 1 tâche
- Traçabilité et visibilité continue



Gestion du code & qualité

Versions

- Validation avant intégration
- Version stable maintenue

Documentation

- Javadoc (backend Spring Boot)
- Documentation API avec Swagger



Spécifications fonctionnelles-Acteurs et rôles

Acteurs

- **Visiteur** : consultation publique
- **Membre** : accès aux documents de son ensemble
- **OWNER** : gestion d'un ensemble
- **ADMIN_GLOBAL** : supervision globale
- Service email : acteur externe (inscription / invitation)

Gestion des rôles

- Rôle lié à l'appartenance à un ensemble
- Un utilisateur peut avoir plusieurs rôles selon l'ensemble
- OWNER / MODERATOR / MEMBER
- Permissions adaptées au contexte



Processus principaux

Principaux cas d'usage

- **Inscription** : formulaire → email de validation → compte actif
- **Création d'ensemble** : créateur devient owner
-
- → **invitations par email** → suivi dans " Gérer les invitation"
-
- → **rattachement à un ensemble** → suivi dans "notification"
- **Gestion des morceaux** : création, ajout de documents
- Consultation : membres consultent morceaux et suppriment leurs documents



Ajout d'un utilisateur

Invitation envoyée

- Deux cas : utilisateur existant / non existant
- Acceptation ou refus
- Notifications automatiques



Ajout d'un morceau

- Vérification du rôle
- Enregistrement
- Notification
- Mise à jour de la liste



Tests unitaires

- ```
@Testvoid
rattacheUtilisateurApresInvitation() { //
 Préparation : utilisateur, ensemble et
 invitation Utilisateur utilisateur = new
 Utilisateur(); Ensemble ensemble = new
 Ensemble(); Invitation invitation = new
 Invitation(); // Appel du service
 InvitationDTO result =
 invitationService.rattacherUtilisateurApres
 Inscription(utilisateur, invitation); //
 Vérifications principales
 assertNotNull(result);
 assertEquals(StatusInvitation.ACCEPTEE,
 invitation.getEtat());}
```
- ce test assure que lorsqu'un utilisateur clique sur une invitation :
- Il est bien rattaché à l'ensemble avec le rôle approprié.
- L'invitation passe à l'état **ACCEPTEE**.
- La méthode du service fonctionne correctement sans dépendre d'autres parties de l'application.



# Tests d'intégration : Authentication

**Objectif** : Il teste l'endpoint `/api/auth/login` dans son ensemble, donc la connexion entre le contrôleur, le service, et éventuellement la base de données (ou son mock).

## Exemple : Authentication

Vérifie que :

- Le serveur répond avec HTTP 200 OK.
- La réponse JSON contient bien un `accessToken`.
- La réponse JSON contient bien un `refreshToken`.

**Outils** : Spring MockMvc, JUnit 5

- Assure que les fonctionnalités critiques (inscription, connexion, déconnexion) sont opérationnelles et sécurisées.



## Rattachement à un ensemble

- `ensemble.getMembres().add(utilisateur)` → simule que l'utilisateur est déjà membre.
- `invitationService.envoyerInvitation(invitation)` → tente d'envoyer l'invitation.
- `assertNull(result)` → vérifie qu'aucune invitation n'est créée.
- `verify(emailService, never()).envoyerInvitation(any())` → vérifie qu'aucun email n'est envoyé.

**Ce test assure que la logique métier bloque l'invitation pour un utilisateur déjà inscrit.**



# Difficultés rencontrées



## Contexte

- Mélange des responsabilités dans le frontend (logique + API + affichage)
- Calcul des rôles partiellement côté frontend → incohérences et risque sécurité

## Déclic important

Le rôle dépend de la relation Utilisateur ↔ Ensemble, pas de l'utilisateur seul.

## Impossible de supprimer un Ensemble avec des enfants :

- La base bloquait la suppression → erreurs d'intégrité

## Solution

- Ajouter CascadeType.ALL sur les collections côté parent (Ensemble)
- Ajouter orphanRemoval = true

# Sécurité de l'application

## Authentification sécurisée

- Création de compte avec mot de passe haché BCrypt (OWASP)
- Utilisation de tokens JWT stateless : access + refresh tokens
- Validation systématique du token côté serveur

## Gestion des sessions

- Tokens dans l'en-tête HTTP : Authorization: Bearer <token>
- Pas de stockage d'état serveur → moins de risques de détournement
- 

## Contrôle d'accès par rôle

- Rôles : OWNER, MODERATOR, MEMBER, ADMIN\_GLOBAL
- Vérification des permissions côté backend avant actions sensibles



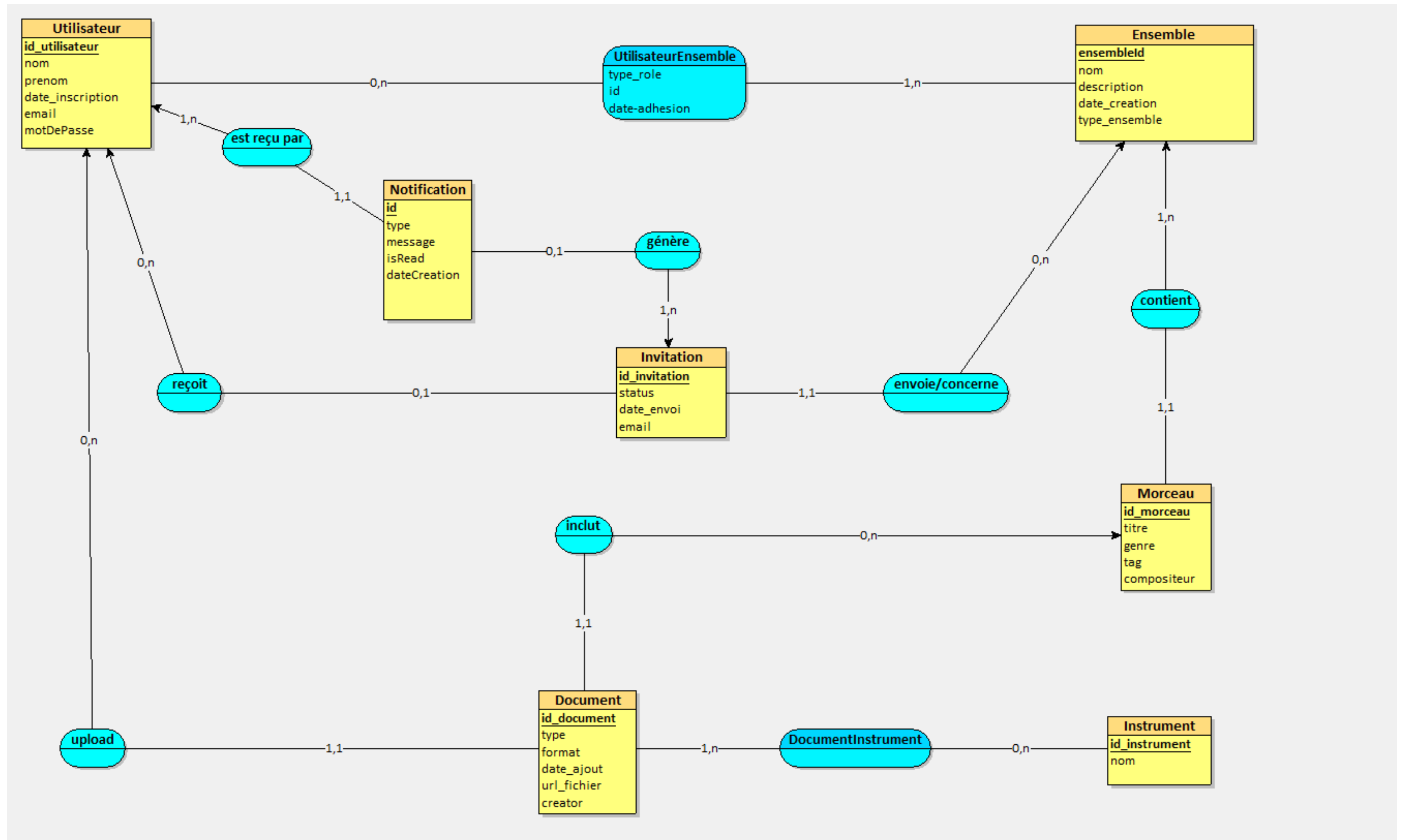
# Optimisation des performances

**N+1**

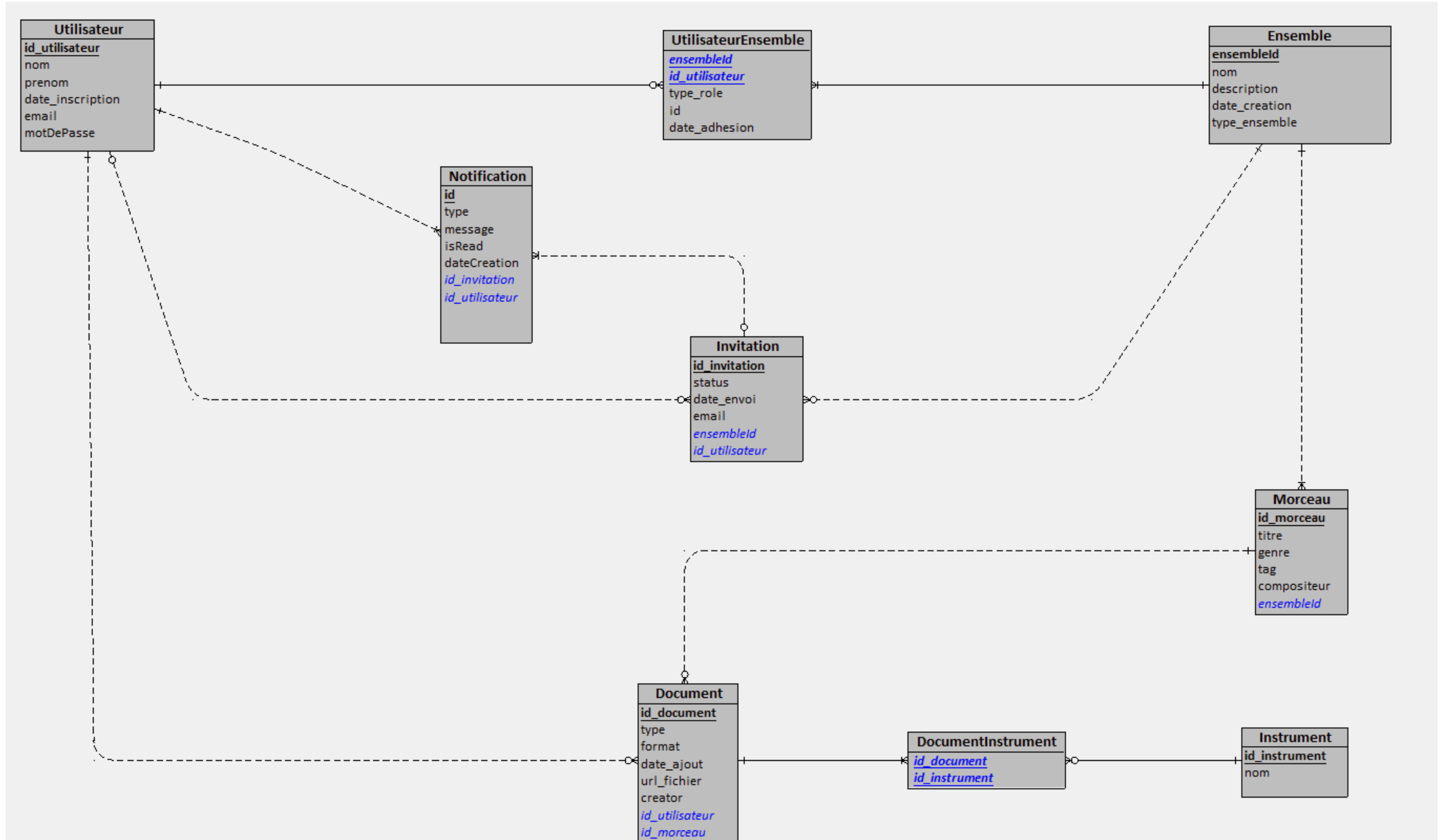
## Contexte

- Certaines entités sont fortement liées : notifications, invitations, utilisateurs
- Si beaucoup d'éléments sont récupérés, Hibernate pourrait générer trop de requêtes (problème N+1)
- Mesures préventives
- Prévoir JOIN FETCH pour récupérer plusieurs relations en une seule requête

# MCD



# MLD





# Prise de recul sur l'architecture

**Objectif :** équilibre entre robustesse, simplicité et évolutivité pour une association musicale.

## Architecture choisie :

- Monolithique multicouche → cohérence globale et maintenabilité.
- Spring Boot → REST, sécurité et accès aux données faciles.
- Microservices écartés → complexité non justifiée : les entités de mon projet sont fortement liées
- Authentification JWT stateless → sessions simplifiées et scalabilité adaptées aux API REST

# Choix techniques & impact

- **JWT stateless** → sécurise les endpoints et simplifie la scalabilité
- **rôles** → contrôle des accès et maintenance simplifiée
- **CascadeType.ALL + orphanRemoval** → supprime les entités liées, cohérence assurée
- **EAGER / LAZY loading** → optimisation des performances et prévention du N+1
- **DTO / Java Records** → protège les données sensibles et rend le code plus clair
- **Docker** → déploiement sûr et reproductible



# Architecture Frontend

- App → composant principal, centralise le routage et la structure
- BrowserRouter → navigation SPA
- NotificationProvider → notifications accessibles partout
- Header / Footer → persistants sur toutes les pages
- <main> → zone dynamique pour les pages



# Pages et fonctionnalités

page creer un ensemble musical\_hd

Choral Riff

Créer un ensemble musical

Nom

Date de création

Instruments/pupitres

Trompette

Claquette

Soprano

Piano

+

Créer ensemble

Annuler

Liens utiles   Contacts   Mentions légales

**Pages principales** : Accueil, Inscription, Connexion, Dashboard

**Gestion des ensembles** : liste, détails, ajout

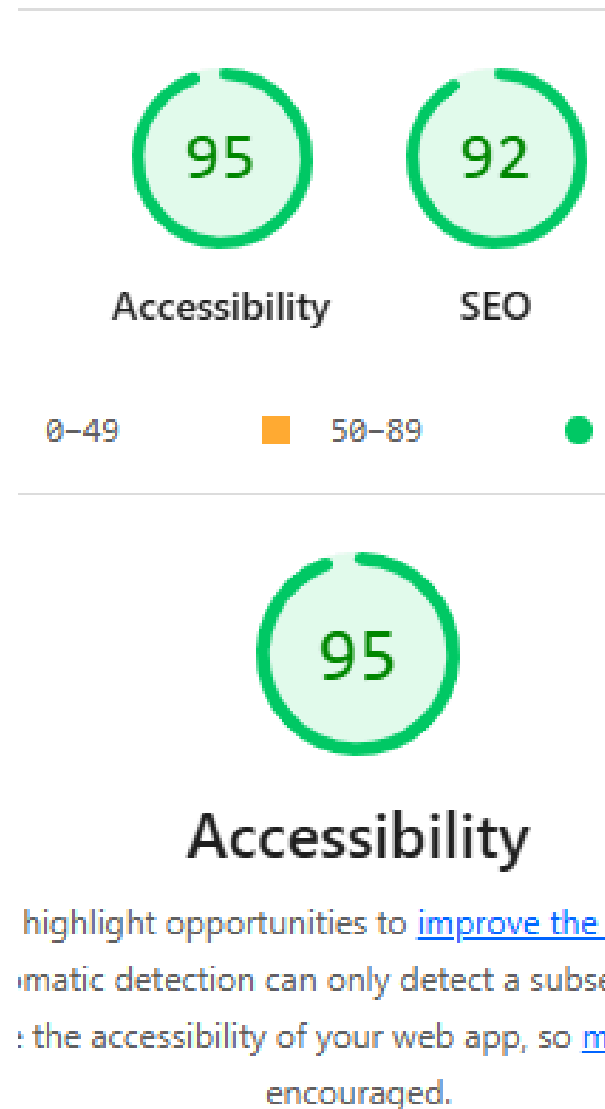
**Gestion des morceaux** : ajout, détails

**Gestion des membres & notifications**

**Collaboration backend/frontend** : endpoints adaptés, DTO cohérents



# Design et accessibilité



- **Maquettes & zoning** → structure des pages, référence pour composants
- **HD mobile** → typographie, positionnement, lisibilité optimisée
- **Palette graphique** → tons pastel, ambiance collaborative et rassurante



## Accessibilité :

- Score Lighthouse 95/100 (WCAG AA)
- HTML sémantique, landmark <main>, labels et ARIA corrects
- Boutons larges et espacés, contraste suffisant, taille de police adaptée
- Navigation intuitive pour public senior

# Veille technologique : sécurité et bonnes pratiques



## Objectif

- Garantir sécurité et intégrité des données
- Suivre recommandations OWASP 2021

## Mesures mises en place

- Authentification sécurisée : BCrypt + JWT stateless
- Contrôle d'accès centralisé : validation côté backend, rôles OWNER/MEMBER/ADMIN
- Protection contre les injections SQL : JPA/JPQL avec paramètres liés
- Isolation via Docker : conteneurs sécurisés, variables d'environnement
- Veille sur dépendances : réduire risques de vulnérabilités



# Veille technologique : CI/CD et industrialisation

| <u>Fonctionnalité</u>         | <u>Responsable</u> | <u>Statut</u>                   |
|-------------------------------|--------------------|---------------------------------|
| Backend API REST              | Zéganadin Nadine   | Auth, rôles, rattachement users |
| Modélisation DB               | Zéganadin Nadine   | MCD/MLD, relations Many-to-Many |
| Frontend (React)              | Binôme             | Intégration maquettes           |
| Tests unitaires / intégration | Zéganadin Nadine   | Endpoints Auth, logique métier  |
| CI/CD automatisé              | DevOps             | En préparation                  |
| Déploiement prod              | DevOps             | À mettre en place               |

# Conclusion et perspectives



## Bilan du projet

- Mise en pratique complète du rôle de Concepteur Développeur d'Applications
- Compétences mobilisées : architecture sécurisée, logique métier, choix techniques justifiés
- Objectif fonctionnel : faciliter la collaboration et l'organisation des choristes à distance

## Perspectives d'amélioration

- Conformité RGPD : suppression de compte et droit à l'effacement
- Annotations de partitions pour enrichir la pratique musicale
- Espace personnel pour Owner : gestion des fichiers et indicateur isOriginal
- Lecture avancée des partitions et fichiers audio/vidéo
- Sécurité renforcée : validation backend, protection des données sensibles
- Pré-autorisation côté controller via @PreAuthorize Spring Security
- Poursuite du projet au-delà de la priorisation MoSCoW